

Dependency Injection, IoC & Loosely Coupled Design

1 Why Spring Core?

- Java already has: Classes, Objects, Constructors, Interfaces, Inheritance, Packages
- **Spring's real job** → create objects, manage dependencies, connect objects cleanly
- As apps grow, one class depends on another, which depends on another → gets messy
- 👉 **Spring Core solves this problem**

2 What is a Dependency?

┆ A dependency = something a class **needs** to do its job.

Example:

OrderService needs EmailService to send an email after placing an order

```
java
private EmailService emailService = new EmailService();
```

➔ Here `OrderService` is **creating its own dependency** — this is the root problem.

3 Tight Coupling vs Loose Coupling

Tight Coupling	Loose Coupling
Hard to change	Easy to change
"I can only travel by THIS car"	"I need transportation — car/bus/train"
Depends on one fixed class	Depends on an idea/interface

Tight coupling in code:

OrderService —directly creates—> EmailService

✗ If requirement changes (Email → SMS), we must **edit OrderService** itself.

┆ 🗝 **Tight Coupling = one class directly depends on a specific concrete class**

4 Step 1 Fix: Use an Interface

```
java
```

```
interface NotificationService {
    void sendNotification();
}

class EmailService implements NotificationService { ... }
class SmsService implements NotificationService { ... }
```

Now `OrderService` depends on the **interface**, not a fixed class. ✅ Better But...

```
java
private NotificationService notificationService = new EmailService();
```

❌ Still creating the object itself with `new` → still deciding the implementation

💡 Creating objects isn't the problem. Creating them in the **WRONG** place is.

`OrderService` should only do order logic — **not decide which notification class to use**. This breaks:

- S → Single Responsibility Principle
- O → Open-Closed Principle

5 The Real Fix: Dependency Injection (DI)

Instead of creating the dependency *inside*, we **receive it from outside**.

```
java
class OrderService {
    private NotificationService notificationService;

    // dependency passed in via constructor
    public OrderService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    public void placeOrder() {
        System.out.println("Order placed successfully");
        notificationService.sendNotification();
    }
}
```

Now `OrderService` just says:

“Give me *something* that can send a notification.”

Object creation moves to `Main`:

```
java
```

```
NotificationService ns = new EmailService();
OrderService os = new OrderService(ns);
```

★ Definition

Dependency Injection = a class *receives* the objects it needs from outside, instead of creating them itself.

Remember it as:

- "Don't create your dependency. Receive it."
- "Ask for what you need, don't build it yourself."

🧠 Note: DI is a general design principle (works in plain Java too) — **Spring did not invent it, Spring just automates it.**

6 Benefits of DI

Benefit	Why
✅ Easy to change	Swap <code>EmailService</code> → <code>SmsService</code> without touching <code>OrderService</code>
✅ Easier to test	Use a <code>FakeNotificationService</code> for testing, no real email/SMS sent
✅ Reusable	Same <code>OrderService</code> works with Email, SMS, WhatsApp, Fake, etc.

7 Types of Dependency Injection

◆ 1. Constructor Injection

Dependency passed via constructor. ✅ Preferred — makes required dependency clear.

```
java
```

```
public OrderService(NotificationService ns) {
    this.notificationService = ns;
}
```

◆ 2. Setter Injection

Dependency passed via a setter method. Useful when dependency is **optional** or changeable later.

```
java
```

```
public void setNotificationService(NotificationService ns) {
    this.notificationService = ns;
}
```

◆ 3. Field Injection

(Covered later in Spring — uses annotations directly on fields)

8 What is IoC (Inversion of Control)?

Before DI: `OrderService` controlled object creation (control inside the class) **After DI:** object created outside, given to `OrderService` (control moved outside)



🔑 This **shift of control from inside → outside** the class = **Inversion of Control**

Simple definition:

- Earlier → the class **created** what it needed
- Now → the class **receives** what it needs

9 IoC vs DI — Relationship

IoC	DI
The principle/idea	The technique to achieve it

IoC is the goal. Dependency Injection is *one way* to achieve that goal.

10 Where Does Spring Fit In?

In plain Java, `Main` manually creates & wires every object. ✅ Fine for small apps ❌ Messy for large apps (hundreds of classes)

So we need a system that automatically:

- Creates objects
- Manages objects
- Connects (wires) objects together

➡ This is the Spring IoC Container

1 1 Spring Core — 3 Main Jobs

1. Creates objects
2. Manages objects
3. Connects objects together

This is the **foundation of Spring**, before learning Spring MVC/Boot/JPA/Security.

1 2 What is a "Bean"?

- Normal Java → we call it an **object**
- Spring → an object **created & managed by Spring** = a **Bean**

! Every Spring bean is an object, but **not every object is a Spring bean**.

```
java
EmailService emailService = new EmailService(); // normal object
```

If **Spring** creates & manages it → it becomes a **Bean** ✓

1 3 Manual Java Flow vs Spring Flow

Plain Java:

```
Main → creates → EmailService
Main → creates → OrderService
Main → injects EmailService into OrderService
```

Spring:

```
Spring IoC Container → creates → EmailService (Bean)
Spring IoC Container → creates + injects → OrderService (Bean)
```

✓ FINAL SUMMARY (Quick Revision)

- **Dependency** = something a class needs
- **Tight Coupling** = class directly depends on a concrete class ✗
- **Dependency Injection (DI)** = receive dependency from outside instead of creating it ✓
- **IoC (Inversion of Control)** = control of object creation moves from class → external system
- **DI is one way to achieve IoC**

- **Spring IoC Container** = automates object creation, management & wiring
- **Bean** = an object managed by Spring

| 🎯 Golden Rule: **"Dependency is normal. Tight dependency is the problem."**